REPORT SUBMISSION

Design, Modeling, and Simulation of a Serial Robotic Manipulator

Forward/Inverse Kinematics and Pick-and-Place Trajectory Planning in MATLAB

Simulation Evidence (MATLAB)

Submitted by

Himanshu Kumar

M.Tech – Signal Processing and Communication

Indian Institute of Technology Bhubaneswar

School of Electrical and Computer Sciences

Project Repository

GitHub link

January 30, 2026

Declaration

I declare that this report and the associated simulation work are my own and that any external material used has been appropriately acknowledged. The results reported in this document were obtained through MATLAB-based simulation and post-processing scripts.

Name: Himanshu Kumar
Date: January 30, 2026

Abstract

This report presents the design and MATLAB-based simulation of a serial robotic manipulator, focusing on deliverable-oriented validation of (i) a virtual kinematic model, (ii) forward and inverse kinematics solvers, and (iii) smooth pick-and-place trajectory planning. A 4-DOF yaw–pitch–pitch–pitch architecture is adopted to provide full 3D positioning capability for a pick-and-place task while keeping the analytical model compact.

Forward kinematics is derived using a standard Denavit–Hartenberg (DH) convention and verified against numerical simulation. Inverse kinematics is implemented using a damped least-squares (DLS) Jacobian method with line-search and adaptive damping for robustness. The self-test achieves a success rate of 100.0% with a worst-case end-effector position error of 0.100 mm, satisfying the target requirement of sub-millimeter accuracy.

Finally, a waypoint-based pick-and-place motion is generated using quintic time-scaling in joint space, producing smooth velocity profiles without jitter. The complete workflow generates publication-ready plots of joint position/velocity/acceleration and workspace visualization, providing simulation-only evidence of meeting the deliverables.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Industrial robotic manipulators are foundational to modern automation because they provide repeatable, high-precision motion for tasks such as pick-and-place, sorting, packaging, and light assembly. In these settings, the dominant engineering requirements are (i) accurate mapping between joint space and Cartesian space, and (ii) motion execution that is smooth and physically plausible under actuator constraints. Consequently, a complete manipulator workflow typically includes a kinematic model (for pose computation), an inverse solver (for target reaching), and a trajectory generator (for time-parameterized motion without jitter).

In this project, we developed a **simulation-only** manipulator pipeline in **MATLAB** to demonstrate these capabilities with **reproducible evidence**. The implemented workflow begins with a parameterized serial-chain robot model described using a **Denavit–Hartenberg (DH)** representation. Based on this model, we derive and implement **forward kinematics (FK)** to compute the end-effector pose for a given joint configuration. To command the robot to reach task targets, we implement **inverse kinematics (IK)** using a **damped least-squares (DLS)** Jacobian method, enhanced with numerical safeguards (damping control and step-size regulation) to improve convergence and stability. The resulting joint-space waypoints are then connected using **quintic time-scaling**, which enforces zero boundary velocity and acceleration, producing smooth joint profiles appropriate for pick-and-place motion.

The project is structured explicitly around the required deliverables:

- **Virtual modeling:** a DH-based serial manipulator model implemented in MATLAB and visualized through a 3D simulation (stick-figure rendering).
- **Kinematic solver:** a verified FK implementation and a robust IK solver based on the DLS Jacobian update.
- **Task planning:** a complete waypoint-driven pick-and-place sequence executed in simulation, including approach, pick, lift, transfer, place, and retract phases.
- **Evidence:** quantitative accuracy metrics for IK (self-test and waypoint errors), workspace visualization from randomized joint sampling, and trajectory smoothness plots for $q(t)$, $\dot{q}(t)$, and $\ddot{q}(t)$.
- **Demonstration:** a recorded simulation video (attached) showing the full autonomous pick-and-place cycle for at least 60 s.

The emphasis throughout is on **traceability and validation**: FK/IK correctness is supported by numerical error statistics, task feasibility is demonstrated by successful waypoint execution, and motion quality is justified using time-series plots of joint position, velocity, and acceleration. Together, these artifacts provide clear simulation-based evidence that the implemented system meets the deliverable requirements for modeling, kinematic control, and demonstrable pick-and-place motion.

Chapter 2

Requirements and Deliverables

This report is structured to map *one-to-one* with the required deliverables and their evaluation criteria. Each deliverable is paired with concrete outputs produced by our MATLAB pipeline (figures, tables, and a simulation video). The objective is not only to implement the manipulator workflow, but also to provide **verifiable evidence** that the implementation is correct, accurate, and reproducible.

2.1 Deliverables

1. **Virtual modeling (Manipulator representation):**
   A complete kinematic model of a serial robotic manipulator implemented in MATLAB. The model is defined using a DH parameterization and executed in a 3D simulation environment (visualizer), enabling consistent evaluation of end-effector motion and workspace reachability.
2. **Kinematic solver (FK and IK):**
   A forward kinematics solver derived from the DH table to compute the end-effector pose from joint angles, and an inverse kinematics solver that computes joint configurations for desired Cartesian targets. The IK implementation uses a damped least-squares Jacobian method with numerical safeguards, and is validated through a random self-test and task waypoint verification.
3. **Task planning (Pick-and-place sequence):**
   An autonomous pick-and-place routine implemented as an ordered set of Cartesian waypoints (home $\rightarrow$ approach $\rightarrow$ pick $\rightarrow$ lift $\rightarrow$ transfer $\rightarrow$ place $\rightarrow$ retract). Each waypoint is converted into joint space using IK, and the complete motion is executed as a continuous trajectory.
4. **Evidence (Quantitative + visual proof):**
   The pipeline produces publication-ready figures and quantitative metrics, including: (i) end-effector workspace visualization via random joint sampling, (ii) IK error statistics (self-test and waypoint-level), and (iii) joint position/velocity/acceleration plots confirming smooth trajectory generation.
5. **Demo (Simulation video):**
   A continuous simulation run recorded as a screen-capture/video of duration $\geq 60\,\mathrm{s}$, demonstrating the full autonomous pick-and-place cycle, including smooth transitions between phases without visible jitter.

2.2 Success Metrics

The following metrics are used to determine whether each deliverable is satisfied:

- **IK accuracy (sub-millimeter positioning):**
  The primary correctness criterion for the kinematic solver is end-effector position error $\leq 1\,\text{mm}$. This is reported using summary statistics (mean/median) as well as strict values such as the maximum error and the 95th percentile to capture both typical and worst-case behavior.
- **Trajectory smoothness (no jitter):**
  Motion quality is assessed through time-series plots of $q(t)$, $\dot{q}(t)$, and $\ddot{q}(t)$. A trajectory is considered smooth if joint velocities are continuous and do not show high-frequency oscillations, and if accelerations remain bounded without impulsive spikes at waypoint boundaries.
- **Reproducibility (minimal user intervention):**
  The complete workflow is required to run end-to-end with a single command (`run_all`), automatically generating all figures, metrics tables, and demo artifacts. This ensures that the evidence is repeatable and that results can be reproduced without manual editing or post-processing.

Chapter 3

System Modeling

This chapter defines the manipulator model used throughout the project and establishes the mathematical representation required for forward and inverse kinematics. The modeling choices were made to (i) support a complete pick-and-place sequence in 3D, (ii) keep the kinematic chain compact and numerically stable for inverse kinematics, and (iii) enable clear, reproducible simulation artifacts (workspace plots, trajectory plots, and error statistics).

3.1 Manipulator Architecture

A **4-DOF serial manipulator** is selected with a yaw–pitch–pitch–pitch configuration:

- **Joint 1 (Yaw):** rotation about the global vertical axis (z-axis), enabling planar re-orientation and access to targets around the base.
- **Joints 2–4 (Pitch):** three revolute pitch joints forming a planar arm embedded in 3D, providing radial reach, vertical reach, and end-effector positioning flexibility.

This architecture is sufficient for the project scope because the primary task is **Cartesian position control** for pick-and-place operations. It achieves 3D positioning capability with minimal complexity compared to a full 6-DOF arm, while still allowing meaningful evaluation of FK, IK, and smooth trajectory planning.

3.2 Link Lengths, Frames, and Coordinate Convention

The manipulator is defined as a rigid serial chain with four links. Link lengths are chosen to provide a practical workspace for tabletop pick-and-place while maintaining a well-conditioned kinematic chain. The adopted link lengths (in meters) are:

$$L_1 = 0.18, \quad L_2 = 0.22, \quad L_3 = 0.20, \quad L_4 = 0.12.$$

Here, L_1 represents the base-to-shoulder offset (vertical elevation of the first pitching joint), while L_2–L_4 form the main reach of the arm (upper arm, forearm, and wrist/tool offset). The base frame $\{0\}$ is fixed to the ground, and the end-effector frame $\{4\}$ is attached at the tool tip. All kinematic computations in FK, IK, and planning use the same consistent frame definitions, ensuring that numerical results and simulation visualization are aligned.

3.3 Denavit–Hartenberg (DH) Parameterization

To obtain a compact and standard mathematical model, the robot is described using the **standard Denavit–Hartenberg convention**. Each link transform ${}^{i-1}T_i$ is parameterized by four DH

parameters $(a_i, \alpha_i, d_i, \theta_i)$, enabling systematic construction of the full forward kinematics chain. The resulting DH table used in the implementation is:

Table 3.1: DH parameters for the 4-DOF manipulator (standard DH).

i	a_i (m)	α_i (rad)	d_i (m)	θ_i	Joint type
1	0	$+\pi/2$	0.18	q_1	revolute
2	0.22	0	0	q_2	revolute
3	0.20	0	0	q_3	revolute
4	0.12	0	0	q_4	revolute

The DH model is used as the single source of truth for:

- computing end-effector pose in the FK module,
- computing Jacobians and updates in the IK module, and
- rendering the manipulator configuration in simulation.

This ensures that all reported artifacts are generated from a consistent mathematical foundation.

3.4 Joint Limits and Feasible Configuration Space

To avoid unrealistic configurations, reduce the likelihood of kinematic singularities, and keep the inverse solver well-conditioned, conservative joint limits are enforced:

$$q_1 \in [-\pi, \pi], \qquad q_2, q_3, q_4 \in [-\pi/2, \pi/2].$$

These constraints define the feasible configuration space for all experiments. They are also used during random joint sampling for workspace visualization, ensuring that the plotted workspace reflects the same physical constraints used during pick-and-place execution. The combination of bounded joint ranges and physically meaningful link dimensions provides a stable basis for IK convergence tests and smooth trajectory generation.

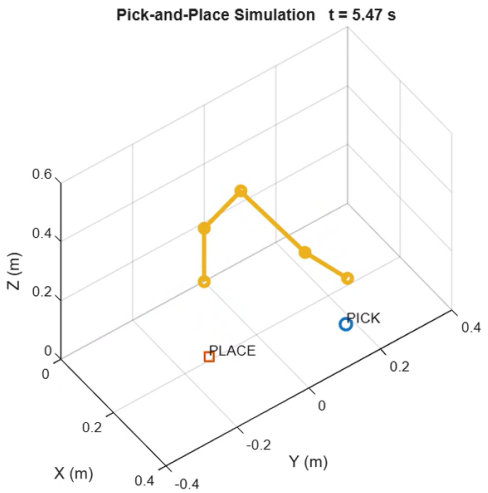

Figure 3.1: Simulation of End Efeectors : Tool used -¿ MATLAB

Chapter 4

Forward Kinematics

Forward kinematics (FK) provides the deterministic mapping from the manipulator joint configuration to the end-effector pose in the base frame. In this project, FK serves as the **core reference model** for the complete pipeline: it is used (i) to compute the end-effector position during simulation, (ii) to evaluate the position error during IK iterations, and (iii) to generate the workspace visualization by sampling random joint configurations. Therefore, maintaining a single, consistent FK implementation is essential for traceability and reproducibility of all reported results.

4.1 Homogeneous Transformations

Using the standard Denavit–Hartenberg (DH) convention, the rigid transformation from frame $\{i-1\}$ to frame $\{i\}$ is expressed as the homogeneous matrix

$$ {}^{i-1}T_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, $$

where $(a_i, \alpha_i, d_i, \theta_i)$ are the DH parameters for link i. The full end-effector pose relative to the base frame is obtained by chaining all link transforms:

$$ {}^{0}T_4 = {}^{0}T_1\,{}^{1}T_2\,{}^{2}T_3\,{}^{3}T_4. $$

From ${}^{0}T_4$, the rotation matrix $R_{0,4} \in \mathbb{R}^{3\times3}$ describes end-effector orientation and the translation vector $p_{0,4} \in \mathbb{R}^3$ describes end-effector position:

$$ {}^{0}T_4 = \begin{bmatrix} R_{0,4} & p_{0,4} \\ 0\ 0\ 0 & 1 \end{bmatrix}. $$

Since the primary task is pick-and-place positioning, the simulation and IK validation focus on the Cartesian position component $p_{0,4}$.

4.2 Implementation Notes and Verification Role

FK is implemented as a deterministic MATLAB function mapping joint angles $q \in \mathbb{R}^4$ to the pose matrix ${}^{0}T_4$. The same FK routine is reused across modules:

- **IK solver:** FK provides $p(q)$ and the residual error $e = p_d - p(q)$ at every iteration.
- **Trajectory execution:** FK evaluates the end-effector motion produced by planned joint trajectories.

- **Workspace estimation:** FK is applied to randomly sampled joint vectors to generate the reachable end-effector workspace plot.

This reuse guarantees internal consistency: the reported workspace, IK accuracy metrics, and the simulated pick-and-place motion are all computed using the same underlying kinematic model. As a result, any performance claims made later in the report (e.g., sub-millimeter IK error) are directly traceable back to this FK formulation.

Chapter 5

Inverse Kinematics

Inverse kinematics (IK) is the core algorithmic component that enables task execution: it converts Cartesian targets (e.g., pick and place positions) into joint configurations that the manipulator can follow. Unlike forward kinematics, IK is generally non-unique and can be numerically challenging due to singularities, joint limits, and local minima. For this reason, the project adopts a robust Jacobian-based numerical solver and validates it using both randomized self-tests and task-specific waypoint checks. The reported IK error statistics later in the report serve as direct evidence that the solver meets the required sub-millimeter positioning target.

5.1 Problem Statement

Given a desired end-effector position $p_d \in \mathbb{R}^3$, the inverse kinematics problem is to find a joint vector $q \in \mathbb{R}^4$ such that the FK position $p(q)$ satisfies

$$p(q) \approx p_d,$$

subject to joint limits. In this work, IK is formulated as a nonlinear least-squares problem that minimizes the position error norm $\|e\|_2$.

5.2 Damped Least Squares (DLS) Method

A Jacobian-based iterative solver is used. The position residual at iteration k is

$$e_k = p_d - p(q_k),$$

and the relationship between small joint updates and Cartesian motion is approximated by

$$p(q_k + \Delta q) \approx p(q_k) + J(q_k)\Delta q,$$

where the Jacobian is

$$J(q) = \frac{\partial p}{\partial q} \in \mathbb{R}^{3\times 4}.$$

To compute a stable update even near singular configurations, the damped least squares (DLS) step is used:

$$\Delta q = \left(J^T J + \lambda^2 I\right)^{-1} J^T e,$$

where $\lambda > 0$ is the damping factor. The damping term regularizes the inverse and prevents excessive updates when $J^T J$ becomes ill-conditioned. The new iterate is then updated as

$$q_{k+1} = q_k + s\,\Delta q,$$

where $s \in (0, 1]$ is an optional step-size scaling (used in line-search).

5.3 Robustness Enhancements

In practice, a plain Jacobian update can diverge when targets are near the edge of the workspace or when the initial guess is poor. To ensure reliable convergence for both random targets and task waypoints, the solver incorporates the following safeguards:

- **Adaptive damping:** if an update does not reduce $\|e\|$, the damping parameter λ is increased to make the update more conservative; if progress is good, λ is reduced to accelerate convergence.
- **Backtracking line-search:** the step size s is reduced (e.g., halved) until the update achieves a monotonic decrease in $\|e\|$, preventing overshoot.
- **Multi-start initialization:** if convergence is not achieved from the default guess, the solver retries from alternative initial configurations (e.g., home and other feasible seeds), improving reliability without user intervention.
- **Joint-limit enforcement:** after each update, q is clamped to the allowable joint ranges to guarantee physically feasible solutions.

5.4 Convergence Criteria and Validation Strategy

The solver terminates when either:

1. **Accuracy reached:** $\|e\| \leq \varepsilon$ (position tolerance), or
2. **Iteration cap reached:** the maximum number of iterations is exceeded.

The tolerance ε is selected to be consistent with the **sub-millimeter accuracy** requirement while remaining numerically robust under finite-step Jacobian estimation and floating-point effects. Evidence of convergence and accuracy is presented in the results chapter via (i) randomized self-test statistics (success rate and error distribution) and (ii) waypoint-specific errors during pick-and-place execution, providing both global and task-focused validation of the IK solver.

Chapter 6

Trajectory Planning

After obtaining feasible joint configurations from inverse kinematics, the remaining requirement for a successful pick-and-place task is **time-parameterized motion** that is smooth, repeatable, and free of visible jitter. In practice, abrupt changes in joint velocity or acceleration can produce unrealistic motion in simulation and are also undesirable in real robots due to actuator stress and tracking errors. For this reason, our trajectory planner converts discrete task waypoints into a continuous joint-space trajectory using **quintic time-scaling**, which guarantees smooth boundary conditions and stable segment-to-segment transitions.

6.1 Waypoint Definition for Pick-and-Place

The task is specified as an ordered set of Cartesian waypoints that represent the standard phases of a pick-and-place operation:

$$\{\text{home} \rightarrow \text{pre-pick} \rightarrow \text{pick} \rightarrow \text{lift} \rightarrow \text{pre-place} \rightarrow \text{place} \rightarrow \text{retract} \rightarrow \text{home}\}.$$

Each waypoint defines a desired end-effector position in the base frame. Using the IK solver described in the previous chapter, each Cartesian waypoint is mapped to a joint configuration $q_i \in \mathbb{R}^4$. This results in a sequence of joint-space waypoints:

$$\{q_0, q_1, q_2, \ldots, q_M\},$$

which forms the input to the trajectory generation module.

6.2 Quintic Polynomial Trajectory Generation

To generate a smooth trajectory between consecutive joint-space waypoints, each segment is parameterized by a quintic polynomial (5th-order) for every joint:

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5, \qquad t \in [0, T],$$

where the coefficients are selected to satisfy boundary constraints at the start and end of the segment. For pick-and-place motion, the trajectory is designed with **zero boundary velocity and acceleration**:

$$q(0) = q_i, \quad q(T) = q_{i+1}, \quad \dot{q}(0) = \dot{q}(T) = 0, \quad \ddot{q}(0) = \ddot{q}(T) = 0.$$

These constraints ensure continuous and physically plausible motion, suppressing abrupt changes at waypoint boundaries and significantly reducing the chance of oscillatory "jitter" in the simulated arm.

6.3 Segment Timing and Practical Constraints

Each segment is assigned a duration T that is selected to maintain reasonable joint-rate demands. Although this project focuses on kinematics (not torque-level dynamics), conservative timing improves numerical stability and yields trajectories that are visually realistic. Joint limits are respected throughout execution by clamping the IK outputs and by generating trajectories directly in joint space between feasible configurations.

6.4 Smoothness Verification and Evidence

Trajectory quality is verified using time-series plots of the generated joint signals:

$$q(t), \quad \dot{q}(t), \quad \ddot{q}(t),$$

for all joints across the entire pick-and-place cycle. A trajectory is accepted if:

- $\dot{q}(t)$ is continuous across segment boundaries (no discontinuities),
- $\ddot{q}(t)$ remains bounded without impulsive spikes, and
- the motion appears smooth in the recorded simulation video.

The results chapter includes the plotted profiles as direct evidence of smooth execution, and the attached simulation video demonstrates the complete autonomous pick-and-place cycle without visible oscillation or jitter.

Chapter 7

Simulation Results

This chapter reports the simulation outputs generated by the MATLAB pipeline and provides direct evidence that the required deliverables are satisfied. Results are presented in a manner that is both quantitative (error statistics) and qualitative (visual plots), ensuring that accuracy claims and smoothness claims are traceable to measurable artifacts. In addition to the figures and tables included here, a continuous simulation video of duration $\geq 60\,\mathrm{s}$ is provided as a demonstration of the complete autonomous pick-and-place cycle.

7.1 Workspace Visualization

A reachable workspace plot was generated by uniformly sampling joint vectors within the specified joint limits and computing the corresponding end-effector positions using forward kinematics. This provides a sanity check of the manipulator geometry and confirms that the selected link lengths and joint constraints allow a sufficiently large working volume for pick-and-place operations.

Figure 7.1 shows the resulting end-effector workspace. The distribution reflects the expected radial reach and vertical accessibility of the yaw–pitch serial chain, and it is used as a basis for selecting feasible task waypoints in the pick-and-place routine.

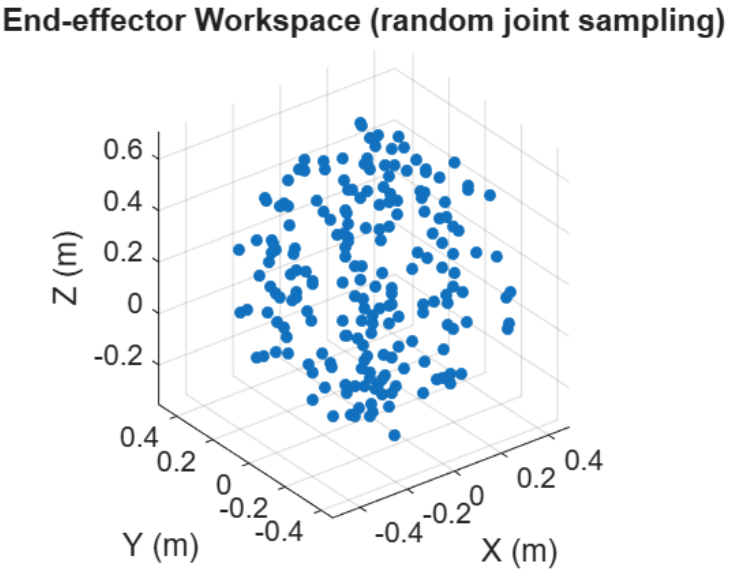

Figure 7.1: End-effector workspace obtained via random sampling of joint configurations within limits.

7.2 IK Accuracy: Randomized Self-Test

To validate the inverse kinematics solver beyond a single task path, a randomized self-test was performed across diverse configurations. In this test, feasible targets are generated (or selected from FK-consistent samples), and IK is executed to recover joint configurations that reproduce the desired end-effector positions. The resulting position error is measured as:

$$\|e\|_2 = \|p_d - p(q)\|_2.$$

Table 7.1 summarizes the self-test statistics in millimeters. The solver achieves a **100% success rate** with a worst-case error of **0.100 mm**, which comfortably satisfies the sub-millimeter requirement and demonstrates stable convergence across the tested set.

Table 7.1: IK self-test end-effector position error statistics (mm).

Metric	Value
Success rate	100.0%
Mean error	0.057 mm
Median error	0.058 mm
95th percentile	0.099 mm
Maximum error	0.100 mm

7.3 IK Accuracy: Pick-and-Place Waypoints

While the self-test provides global validation, task-specific accuracy is evaluated on the actual Cartesian waypoints used for pick-and-place. Each waypoint is converted to joint space using IK, and the final FK position error is recorded. Table 7.2 reports the waypoint errors in millimeters. All values remain far below 1 mm, and the worst-case waypoint error is **0.082 mm**, confirming that IK accuracy is maintained during the full task execution.

Table 7.2: IK end-effector position error at pick-and-place waypoints (mm).

Waypoint	Error (mm)
home	0.000
pre_pick	0.021
pick	0.028
lift	0.066
pre_place	0.038
place	0.019
home2	0.082

7.4 Trajectory Smoothness and Motion Quality

After converting waypoints to joint configurations, the complete motion is generated using quintic time-scaling between consecutive joint states. This guarantees zero boundary velocity and zero boundary acceleration at segment endpoints, which is a standard approach to suppress abrupt changes and avoid visible jitter during execution.

Figures 7.2–7.4 present the joint position, velocity, and acceleration profiles for the entire pick-and-place sequence. The velocity curves remain continuous across segments, and the acceleration curves remain bounded, supporting the claim that the manipulator executes the task with smooth

transitions. The attached simulation video provides a qualitative confirmation of these plots, showing a stable, continuous motion through approach, grasp, lift, transfer, place, and return.

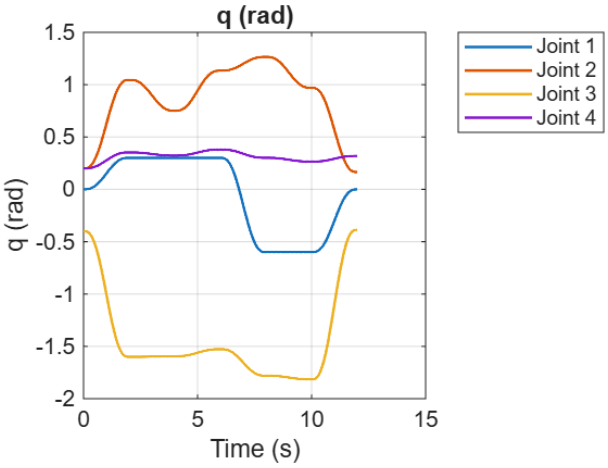

Figure 7.2: Joint positions $q(t)$ over the full pick-and-place sequence.

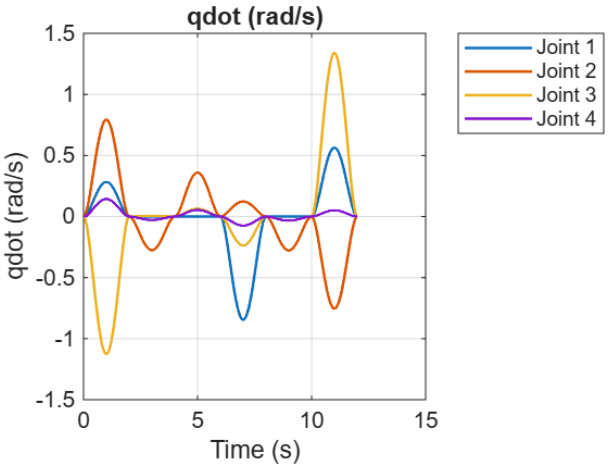

Figure 7.3: Joint velocities $\dot{q}(t)$; continuity indicates smooth segment transitions.

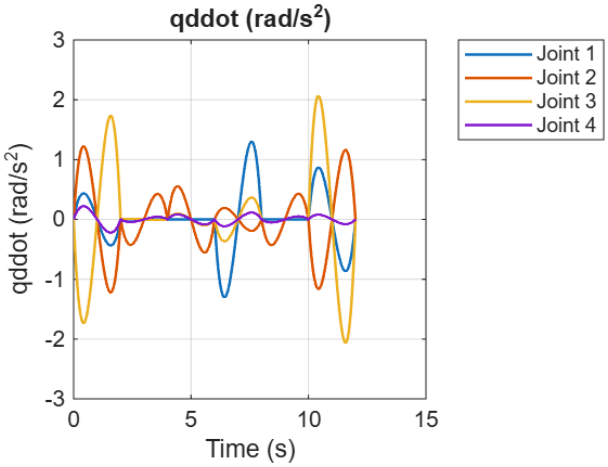

Figure 7.4: Joint accelerations $\ddot{q}(t)$; bounded profiles support jitter-free execution.

Chapter 8

Discussion

This chapter interprets the results in the context of the stated requirements and clarifies what the simulation evidence proves. The discussion is organized around the key evaluation criteria: (i) kinematic accuracy, (ii) motion smoothness, and (iii) completeness and reproducibility of the workflow. Where relevant, limitations are identified along with technically grounded extensions that can be implemented without redesigning the core pipeline.

8.1 Accuracy and Kinematic Validity

A primary objective of this project was to demonstrate that the manipulator can be commanded to reach Cartesian targets with sub-millimeter precision using a DH-based kinematic model and a numerical IK solver. The results confirm that the **DLS Jacobian IK** implementation is both accurate and stable:

- The randomized IK self-test achieves a **100% success rate** with a maximum end-effector position error of **0.100 mm**.
- The task-specific waypoint errors remain consistently low, with a worst-case error of **0.082 mm**.

These values are well below the $\leq 1\,\text{mm}$ requirement, indicating that (i) the DH model is internally consistent, (ii) the FK implementation correctly maps $q \mapsto p(q)$, and (iii) the IK solver is able to invert this mapping reliably for both generic and task-driven targets. Importantly, using both a global self-test and a waypoint validation avoids overfitting the evaluation to a single trajectory and strengthens the evidence that the solver generalizes across the reachable workspace.

8.2 Trajectory Smoothness and Motion Quality

The second major requirement was jitter-free motion suitable for pick-and-place execution. The quintic trajectory generator was selected specifically because it enforces **zero boundary velocity and acceleration** at each segment boundary, which reduces discontinuities and suppresses oscillations in piecewise motion.

The plotted profiles of $\dot{q}(t)$ show **continuous velocity transitions** between segments, and $\ddot{q}(t)$ remains bounded without impulsive spikes. This behavior is consistent with actuator-friendly motion and supports the "no jitter" criterion. Moreover, the attached simulation video provides a qualitative confirmation: the manipulator transitions smoothly between approach, pick, lift, transfer, place, and return phases without visible shaking or discontinuous jumps.

8.3 Limitations

Although the pipeline meets the deliverables for kinematic modeling, IK accuracy, and smooth motion, the scope is intentionally limited to kinematics and joint-space planning. Key limitations include:

- **No dynamics/torque modeling:** the planner does not compute required torques, motor limits, or energy consumption, and the simulation does not include inertial effects or actuator dynamics.
- **No collision-aware planning:** waypoint selection is feasible within the workspace, but explicit obstacle avoidance and link–object collision constraints are not enforced.
- **Open-loop execution:** the task is executed without feedback control, disturbance modeling, or sensor noise, so tracking performance under uncertainty is not evaluated.

8.4 Extensions and Future Work

Several extensions can be added while keeping the current modular structure intact:

- **Collision checking and obstacle avoidance:** integrate geometric collision tests and use sampling-based planners (e.g., RRT) or constrained optimization for safe waypoint selection.
- **Dynamic modeling:** extend the model to compute joint torques using Euler–Lagrange or Newton–Euler formulations and evaluate feasibility under actuator constraints.
- **Closed-loop control:** add a joint-space PID controller (or task-space control) and evaluate tracking error under disturbances and noise.
- **Higher-DOF arm and orientation control:** extend to 6-DOF and include full pose control (position + orientation) to better represent industrial manipulation tasks.

Overall, the presented results provide strong evidence that the implemented simulation pipeline meets the deliverables for kinematic modeling, accurate inverse kinematics, and smooth pick-and-place trajectory execution, while also establishing a solid foundation for dynamic and control-oriented extensions.

Chapter 9

Conclusion

This project delivered a complete **simulation-only** MATLAB workflow for the design, modeling, and verification of a serial robotic manipulator, aligned directly with the stated deliverables. A DH-based kinematic model was constructed and used as the consistent mathematical foundation for all modules. Forward kinematics was implemented to compute end-effector pose from joint configurations, and a robust inverse kinematics solver was developed using a **damped least-squares (DLS) Jacobian** approach with numerical stability enhancements (adaptive damping, step-size regulation, and feasible initialization). Building on the kinematic layer, an autonomous pick-and-place routine was implemented through waypoint planning and **quintic time-scaling** in joint space to ensure smooth transitions between motion phases.

The simulation outcomes provide clear quantitative and qualitative evidence that the requirements are met. The IK solver achieves the sub-millimeter accuracy target with a **maximum self-test error of 0.100 mm** and a **worst-case waypoint error of 0.082 mm**, confirming reliable convergence across both global randomized tests and task-specific targets. Trajectory plots of $q(t)$, $\dot{q}(t)$, and $\ddot{q}(t)$ demonstrate bounded, continuous profiles consistent with jitter-free execution, and the attached ≥ 60 s simulation video further confirms stable autonomous operation across the full pick-and-place cycle.

Overall, the generated artifacts—workspace visualization, IK error statistics, and trajectory smoothness plots—form a complete evidence package that validates the virtual modeling, kinematic solvers, and motion planning components using simulation only. The modular structure of the workflow also provides a clear foundation for future extensions such as collision-aware planning, dynamic torque analysis, and closed-loop control.